UNIVERSITÀ DEGLI STUDI DI PADOVA
CORSO DI LAUREA IN INFORMATICA (L-31)

Corso di Ingegneria del Software
Anno Accademico 2025/2026

Norme di Progetto

Gruppo: NightPRO

swe.nightpro@gmail.com

Data: 2026-03-17
Versione: 3.0

Tabella delle Versioni

Versione	Data	Autore/i	Descrizione delle Modifiche	Verificatore
3.0	2026-03-17	F. Zanella	Approvazione del documento	-
2.1	2026-03-12	S. Perozzo	Ristrutturazione uniforme del documento.	Leonardo Bilato
2.0	2026-02-25	D. Biasuzzi	Approvazione del documento	-
1.4	2026-01-28	D. Biasuzzi	Completate sezioni Analisi dei Requisiti e Codifica; integrato Ciclo di Deming (PDCA) e metriche EVM nella sezione Ottimizzazione dei processi, Gestione dei Rischi, DdB e spiegazione progettazione UI per MVP	Francesco Zanella
1.3	2025-12-03	M. Ogniben	Chiariti compiti Responsabile, aggiunta descrizione nuova sezione verbali interni, sistemata descrizione organizzazione repository	Giovanni Ponso
1.2	2025-11-13	D. Biasuzzi	Aggiunta struttura iniziale paragrafo relativo alla fornitura (sezioni "Analisi Req". e "Codifica" da completare)	Michele Ogniben
1.1	2025-11-11	M. Ogniben	Aggiunto paragrafo relativo ai Processi Organizzativi	Davide Biasuzzi
1.0	2025-11-05	D. Biasuzzi	Modifica struttura del file e verifica	Francesco Zanella
0.1	2025-11-05	G. Ponso; D. Biasuzzi	Creazione documento + redazione processi di supporto	Francesco Zanella

Indice

Tabella delle Versioni 2

1 Informazioni Generali 5
1.1 Componenti del Gruppo . . . 5

2 Introduzione 6
2.1 Scopo del Documento . . . 6
2.2 Glossario . . . 6
2.3 Riferimenti . . . 6
2.3.1 Riferimenti normativi . . . 6
2.3.2 Riferimenti informativi . . . 6

3 Processi Primari 7
3.1 Fornitura . . . 7
3.1.1 Attività 1 – Gestione della comunicazione con il proponente . . . 7
3.1.2 Attività 2 – Redazione e consegna dei documenti di fornitura . . . 7
3.2 Sviluppo . . . 8
3.2.1 Attività 1 – Analisi dei Requisiti$_G$ . . . 8
3.2.2 Attività 2 – Progettazione . . . 9
3.2.3 Attività 3 – Codifica (MVP) . . . 9

4 Processi di Supporto 11
4.1 Gestione della Configurazione . . . 11
4.1.1 Attività 1 – Organizzazione del repository . . . 11
4.1.2 Attività 2 – Automazione CI/CD . . . 11
4.1.3 Attività 3 – Gestione della comunicazione e degli strumenti collaborativi . 12
4.2 Documentazione . . . 13
4.2.1 Attività 1 – Redazione . . . 13
4.2.2 Attività 2 – Verifica . . . 13
4.2.3 Attività 3 – Approvazione e pubblicazione . . . 14
4.2.4 Verbali . . . 14

5 Processi Organizzativi 16
5.1 Gestione dei Ruoli . . . 16
5.1.1 Attività 1 – Assegnazione e rotazione dei ruoli . . . 16
5.2 Gestione delle Comunicazioni e degli Incontri . . . 18
5.2.1 Attività 1 – Comunicazione interna . . . 18
5.2.2 Attività 2 – Comunicazione esterna . . . 18
5.2.3 Attività 3 – Organizzazione delle riunioni interne . . . 18
5.2.4 Attività 4 – Organizzazione delle riunioni esterne . . . 19
5.3 Pianificazione e Tracciamento delle Attività . . . 19
5.3.1 Attività 1 – Gestione del ciclo di vita dei task . . . 19
5.4 Ottimizzazione dei Processi . . . 20
5.4.1 Attività 1 – Applicazione del Ciclo di Deming (PDCA$_G$) . . . 20
5.4.2 Attività 2 – Raccolta e analisi delle metriche . . . 21
5.5 Gestione dei Rischi . . . 21
5.5.1 Attività 1 – Identificazione e analisi dei rischi . . . 21
5.5.2 Attività 2 – Monitoraggio dei rischi . . . 22

5.6 Diario di Bordo 22
5.6.1 Attività 1 – Preparazione e svolgimento 22
5.7 Progettazione dell'Interfaccia Utente 23
5.7.1 Attività 1 – Prototipazione e validazione UI 23
5.8 Sviluppo delle Competenze 23
5.8.1 Attività 1 – Formazione individuale e condivisione delle conoscenze 23

1 Informazioni Generali

1.1 Componenti del Gruppo

Cognome	**Nome**	**Matricola**
Biasuzzi	Davide	2111000
Bilato	Leonardo	2071084
Zanella	Francesco	2116442
Romascu	Mihaela-Mariana	2079726
Ogniben	Michele	2042325
Perozzo	Samuele	2110989
Ponso	Giovanni	2000558

Tabella 1: Componenti del gruppo NightPRO.

2 Introduzione

2.1 Scopo del Documento

Questo documento definisce le Norme di Progetto$_G$ del gruppo NightPRO per il corso di Ingegneria del Software (A.A. 2025/2026). Stabilisce in forma univoca metodo di lavoro, regole redazionali e criteri di gestione dei documenti, garantendo coerenza, tracciabilità e chiarezza operativa. Ogni processo è descritto attraverso la gerarchia uniforme **Processo → Attività → Procedure → Strumenti**. La presente versione costituisce la base metodologica di riferimento e sarà aggiornata con l'avanzare delle attività.

2.2 Glossario

Per garantire chiarezza terminologica e uniformità nella comprensione dei concetti utilizzati, viene redatto un Glossario$_G$ contenente i termini tecnici e potenzialmente ambigui. Alla **prima occorrenza** nel documento, ogni termine seguito dal simbolo $_G$ è definito formalmente nel glossario. Le occorrenze successive non riportano tale marcatura.

2.3 Riferimenti

2.3.1 Riferimenti normativi

I seguenti documenti costituiscono riferimento vincolante per le attività del gruppo. Le regole e le procedure qui definite devono essere conformi ad essi.

- Norme di Progetto (Ultima modifica: 2026-03-12)
- Capitolato C8 – SmartOrder: piattaforma AI per la gestione intelligente degli ordini nella ristorazione (Ultima modifica: 2025-12-12)
- Regolamento del progetto didattico (Ultima modifica: 2025-11-24)

2.3.2 Riferimenti informativi

I seguenti documenti e risorse sono stati consultati a scopo di studio e orientamento metodologico, senza costituire vincolo normativo.

- Glossario (Ultima modifica: 2026-03-12)
- T2: Ciclo di vita del software (Ultima modifica: 2025-11-24)
- T4: Gestione di Progetto (Ultima modifica: 2025-11-24)
- Approfondimenti standard ISO/IEC 12207:1995 (Ultima modifica: 2025-12-12)
- Documentazione ufficiale Git (Ultima modifica: 2026-03-12)
- Documentazione ufficiale GitHub Actions (Ultima modifica: 2026-03-12)
- Documentazione ufficiale Overleaf / LaTeX (Ultima modifica: 2026-03-12)
- Documentazione ufficiale Penpot (Ultima modifica: 2026-03-12)

3 Processi Primari

3.1 Fornitura

3.1.1 Attività 1 – Gestione della comunicazione con il proponente

Procedure

1. Il Responsabile di Progetto contatta il referente di *Ergon Informatica* (Gianluca Carlesso) per concordare data, ora e piattaforma dell'incontro.
2. Prima di ogni riunione, il Responsabile prepara un'agenda con i punti da discutere e la condivide con il gruppo tramite Telegram almeno 24 ore prima.
3. Durante la riunione, un membro designato prende nota delle decisioni e degli impegni assunti.
4. Al termine della riunione, il Responsabile redige il verbale esterno entro 48 ore, secondo la struttura definita nella sezione 4.2.4.
5. Il verbale viene verificato da un secondo membro e, se richiesto dal proponente, sottoposto ad approvazione prima della pubblicazione.
6. Gli impegni e i requisiti emersi vengono trascritti come task su GitHub Projects e assegnati ai membri competenti.

Strumenti

- **Google Meet**: piattaforma per videoconferenze con il proponente.
- **Telegram**: canale per comunicazioni operative rapide con i referenti aziendali.
- **Email** (`swe.nightpro@gmail.com`): comunicazioni formali e invio di documentazione ufficiale.
- **GitHub Projects**: tracciamento degli impegni e dei requisiti emersi.

3.1.2 Attività 2 – Redazione e consegna dei documenti di fornitura

Procedure

1. Il Responsabile identifica i documenti richiesti per la milestone$_G$ corrente (es. Lettera di Presentazione, Preventivo dei Costi).
2. Il redattore assegnato crea o aggiorna il file `.tex` su Overleaf, rispettando la struttura definita nella sezione 4.2.
3. Il Verificatore esamina il documento e segnala eventuali non conformità prima del caricamento su GitHub.
4. Il Responsabile approva il documento nella tabella delle versioni e lo pubblica sul sito tramite la pipeline CI$_G$.
5. In caso di documenti soggetti a firma, si aggiunge il suffisso `_firmato` al nome del file prima della consegna.

Strumenti

- **LaTeX / Overleaf**: redazione collaborativa dei documenti ufficiali.
- **GitHub**: versionamento e archiviazione dei sorgenti `.tex` e dei PDF generati.
- **GitHub Actions**: compilazione automatica e pubblicazione su GitHub Pages.

3.2 Sviluppo

3.2.1 Attività 1 – Analisi dei Requisiti$_G$

Procedure

1. Studiare il capitolato$_G$ e la documentazione fornita dal proponente per comprendere il dominio applicativo.
2. Raccogliere i requisiti tramite incontri col proponente; annotare le esigenze emerse nel verbale esterno.
3. Classificare ogni requisito con il codice strutturato:

   `R[Tipo][Numero].[Sottorequisito]`

   dove **Tipo** è uno tra: `F` (Funzionale), `Q` (Qualità), `V` (Vincolo), `P` (Prestazionale), `S` (Sicurezza).
4. Assegnare a ogni requisito una priorità: **Obbligatorio**, **Desiderabile** o **Opzionale**.
5. Modellare i casi d'uso (Use Case) con diagrammi UML$_G$, identificando ogni UC con il codice:

   `UC[Numero].[Sottocaso]`
6. Per ogni caso d'uso, documentare: identificativo e titolo, attore principale, precondizioni, postcondizioni, scenario principale, estensioni, user story.
7. Costruire le matrici di tracciabilità bidirezionale tra casi d'uso e requisiti.
8. Formalizzare i risultati nel documento *Analisi dei Requisiti*$_G$ e sottoporlo al ciclo di verifica/approvazione.

Esempi di codifica

- `RF1.1` – Requisito Funzionale 1, sottorequisito 1
- `RV02` – Requisito di Vincolo numero 2
- `UC1.1` – Primo sottocaso del caso d'uso 1

Strumenti

- **PlantUML**: creazione dei diagrammi dei casi d'uso.
- **Overleaf / LaTeX**: redazione del documento *Analisi dei Requisiti*.
- **GitHub Projects**: tracciamento dei task di analisi.

3.2.2 Attività 2 – Progettazione

Procedure

1. **Progettazione logica**:
   a. Selezionare lo stack tecnologico (linguaggi, framework$_G$, librerie) motivando le scelte rispetto ai requisiti.
   b. Realizzare un Proof of Concept$_G$ (PoC) per validare la fattibilità tecnica delle scelte adottate.
   c. Documentare l'esito del PoC nel repository dedicato (`SmartOrder-PoC`).
2. **Progettazione di dettaglio**:
   a. Definire l'architettura$_G$ del sistema e i pattern architetturali$_G$ adottati.
   b. Produrre il diagramma UML delle classi e i diagrammi di sequenza per le interazioni principali.
   c. Specificare le interfacce tra i componenti e le dipendenze tra moduli.
   d. Verificare che l'architettura soddisfi tutti i requisiti classificati come Obbligatori.
   e. Sottoporre il documento di progettazione al ciclo di verifica/approvazione.

Strumenti

- **PlantUML / strumento UML concordato**: diagrammi architetturali e di classe.
- **Overleaf / LaTeX**: documentazione della progettazione.
- **GitHub** (`SmartOrder-PoC`): versionamento del Proof of Concept.

3.2.3 Attività 3 – Codifica (MVP)

Procedure

3a. Preparazione del branch

1. Creare un branch dal `main` seguendo la convenzione:
   - `feature/[nome]`: nuove funzionalità.
   - `fix/[nome]`: correzione di bug.
   - `hotfix/[nome]`: correzioni urgenti su codice in produzione.
2. Associare il branch al task corrispondente su GitHub Projects.

3b. Implementazione

1. Scrivere il codice rispettando le convenzioni di stile seguenti:
   - **Indentazione**: 4 spazi; vietato il carattere di tabulazione (`tab`).
   - **Lunghezza righe**: massimo 120 caratteri; suddividere le righe più lunghe in modo logico.
   - **Spaziatura**: riga vuota tra blocchi logici; spazi attorno agli operatori binari; parentesi graffe anche per blocchi di una sola istruzione.
2. Applicare le convenzioni di nomenclatura:

- **Classi e Interfacce**: `PascalCase`$_G$ (es. `OrderProcessor`).
- **Metodi e Funzioni**: `camelCase`$_G$, nomi verbali (es. `validateInput()`).
- **Variabili e Parametri**: `camelCase`, nomi descrittivi (es. `confidenceScore`).
- **Costanti**: `UPPER_SNAKE_CASE` (es. `MAX_RETRY_COUNT`).

3. Documentare ogni componente:
   - Intestazione del file con descrizione, autore e data di creazione.
   - Documentazione di ogni funzione pubblica (descrizione, parametri, valore di ritorno).
   - Commenti inline per logiche complesse o non immediatamente intuibili.
   - Marcatori `TODO` e `FIXME` per segnalare lavori in sospeso.
4. Scrivere test unitari per ogni unità di codice prodotta.

3c. Commit e push

1. Formulare messaggi di commit$_G$ chiari e descrittivi, che spieghino sinteticamente la natura della modifica apportata.
2. Ogni commit deve rappresentare un'unità logica di modifica; evitare commit con modifiche non correlate.
3. Effettuare il push sul branch dedicato.

3d. Pull Request e Merge

1. Aprire una pull request$_G$ verso `main` con descrizione delle modifiche apportate.
2. Associare la pull request al task su GitHub Projects.
3. Il Verificatore esegue la code review: verifica stile, logica, test e conformità alle norme.
4. Risolvere eventuali conflitti prima del merge.
5. Il merge sul branch `main` è consentito solo dopo approvazione del Verificatore.
6. Aggiornare lo stato del task su GitHub Projects a "Completato".

Standard di qualità del codice Il codice prodotto deve rispettare i criteri di: **leggibilità**, **manutenibilità**, **modularità**, **testabilità** ed **efficienza**.

Strumenti

- **Git**: controllo di versione distribuito.
- **GitHub** (`SmartOrder-MVP`): hosting del repository, gestione branch e pull request.
- **GitHub Projects**: tracciamento dei task di sviluppo.

4 Processi di Supporto

4.1 Gestione della Configurazione

4.1.1 Attività 1 – Organizzazione del repository

Procedure

1. Mantenere tre repository distinti all'interno dell'organizzazione GitHub **NightPRO**:
   - **Documentazione**: sorgenti LaTeX, PDF generati, sito GitHub Pages.
   - **SmartOrder-PoC**: codice del Proof of Concept.
   - **SmartOrder-MVP**: codice del Minimum Viable Product.
2. Rispettare la struttura del repository `Documentazione`:

```
.github/workflows/   # Workflow CI (build PDF, deploy sito)
src/                 # File .tex sorgenti dei documenti
docs/                # PDF generati automaticamente
site/                # Codice sorgente sito GitHub Pages
template/            # Template e documenti base
report.md            # Report compilazione automatica
```

3. Caricare direttamente su `main` i documenti verificati; non usare branch intermedi per la documentazione.
4. Per il codice sorgente, usare branch dedicati (`feature/`, `fix/`, `hotfix/`) e procedere tramite pull request.

Strumenti

- **Git**: sistema di controllo versione.
- **GitHub**: piattaforma di hosting, revisione codice e collaborazione.

4.1.2 Attività 2 – Automazione CI/CD

Procedure

1. **Workflow di build LaTeX**:
   a. Il workflow si attiva automaticamente a ogni push su file `.tex` in `src/`.
   b. Il sistema rileva le modifiche rispetto all'ultima build valida e identifica i file da ricompilare.
   c. I PDF obsoleti o privi di sorgente vengono eliminati (esclusi i PDF firmati, gestiti manualmente).
   d. La compilazione avviene tramite `latexmk` in ambiente Docker$_G$.
   e. Viene generato il file `report.md` con l'esito della compilazione e i link ai PDF prodotti.
   f. Solo i file effettivamente aggiornati vengono inclusi nel commit automatico.
   g. In caso di errore di compilazione, la pubblicazione viene bloccata.
2. **Workflow di pubblicazione**:
   a. Si attiva al completamento con successo del workflow di build.
   b. Sincronizza la struttura del sito con la cartella `docs/`.

   c. Include automaticamente nuove versioni e nuovi documenti nell'indice del sito.
   d. Pubblica il contenuto su GitHub Pages senza intervento manuale.

3. **Workflow di controllo qualità**:
   a. Il workflow si attiva automaticamente al completamento del workflow di build, esclusivamente in caso di esito positivo.
   b. è possibile eseguire il workflow manualmente tramite trigger esplicito.
   c. Il sistema esegue controlli di qualità sui file LaTeX, tra cui:
      - indice di leggibilità Gulpease;
      - analisi linguistica tramite LanguageTool;
      - analisi statica del codice LaTeX tramite ChkTeX.
   d. I risultati dei controlli vengono aggregati in un report (`quality_report.md`).
   e. Il report viene salvato come artefatto del workflow con retention configurata.
   f. In caso di esecuzione su pull request, il report viene pubblicato automaticamente come commento.
   g. Il workflow non blocca la pipeline in caso di errori nei controlli, ma li segnala nel report.

Strumenti

- **GitHub Actions**: esecuzione automatica dei workflow di build e deploy.
- **Docker**: ambiente di compilazione LaTeX isolato e riproducibile.
- **Latexmk**: compilatore LaTeX con gestione automatica delle dipendenze.
- **GitHub Pages**: hosting del sito statico di documentazione.
- **LanguageTool**: analisi grammaticale e stilistica.
- **ChkTeX**: analisi statica dei file LaTeX.

4.1.3 Attività 3 – Gestione della comunicazione e degli strumenti collaborativi

Procedure

1. Utilizzare **Telegram** come canale principale per comunicazioni asincrone interne, sfruttando la funzionalità topic$_G$ per separare le conversazioni per argomento (es. documentazione, riunioni, diario di bordo).
2. Utilizzare **Google Meet** per riunioni sincrone: condivisione schermo per attività collaborative, link di accesso comunicato su Telegram prima dell'incontro.
3. Utilizzare **GitHub Projects** per la gestione delle attività: ogni task deve avere titolo, descrizione, assegnatario, priorità e stima temporale.

Strumenti

- **Telegram**: messaggistica asincrona interna.
- **Google Meet**: videoconferenze sincrone.
- **GitHub Projects**: pianificazione e tracciamento dei task.

4.2 Documentazione

4.2.1 Attività 1 – Redazione

Procedure

1. Creare o aprire il file sorgente `.tex` su Overleaf$_G$, usando il template condiviso nel repository.
2. Rispettare la struttura obbligatoria di ogni documento:
   - Frontespizio$_G$ con dati e metadati ufficiali.
   - Tabella delle versioni e delle modifiche.
   - Indice dei contenuti.
   - Corpo del documento articolato in sezioni e sottosezioni.
   - (Se necessario) Appendici, glossari, tabelle e riferimenti.
3. Nominare i file secondo la convenzione `snake_case`$_G$:
   - Versione: `vX.Y` — Data: formato ISO `AAAA-MM-GG`.
   - Esempi: `norme_di_progetto_v2.1.tex`, `verbale_interno_2025-10-29.tex`.
4. Marcare la prima occorrenza di ogni termine tecnico con il simbolo $_G$ in apice.
5. Aggiornare la tabella delle versioni a ogni modifica significativa, indicando autore, data e descrizione sintetica delle modifiche.

Strumenti

- **LaTeX**: formato sorgente di tutta la documentazione ufficiale.
- **Overleaf**: editing collaborativo online dei documenti LaTeX.

4.2.2 Attività 2 – Verifica

Procedure

1. Il Verificatore esamina il documento su Overleaf prima del caricamento su GitHub, controllando:
   - conformità alle Norme di Progetto (struttura, nomenclatura, glossario);
   - correttezza dei contenuti e coerenza interna;
   - qualità formale e lessicale;
   - aggiornamento della tabella delle versioni.
2. Il Verificatore segnala al redattore le non conformità riscontrate.
3. La verifica è superata quando tutte le non conformità sono state risolte.
4. Il Verificatore aggiorna il campo “Verificatore” nella tabella delle versioni.

Strumenti

- **Overleaf**: revisione del documento prima del push.
- **Telegram**: canale per la comunicazione delle non conformità al redattore.

4.2.3 Attività 3 – Approvazione e pubblicazione

Procedure

1. **Approvazione** (solo per documenti ufficiali: Lettera di Presentazione, Valutazione Capitolati, Preventivo dei Costi, Norme di Progetto, ecc.):
   a. Il Responsabile esamina il documento verificato e ne certifica la completezza e correttezza.
   b. Il Responsabile aggiorna la tabella delle versioni con la voce di approvazione.
2. **Caricamento su GitHub**:
   a. Il redattore esegue il push del file `.tex` sul branch `main` del repository `Documentazione`.
   b. La pipeline CI si avvia automaticamente e compila il PDF.
   c. Verificare l'esito della build nel file `report.md`.
3. **Documenti firmati**: aggiungere il suffisso `_firmato` (o `_signed`) al PDF prima della consegna (es. `norme_di_progetto_v2.1_firmato.pdf`).
4. **Pubblicazione**: al termine della build con esito positivo, il workflow di deploy aggiorna automaticamente il sito GitHub Pages.

Strumenti

- **GitHub**: repository e storico delle versioni.
- **GitHub Actions**: compilazione e deploy automatici.
- **GitHub Pages**: portale di consultazione pubblica della documentazione.

4.2.4 Verbali

Procedure

1. Strutturare ogni verbale nelle seguenti sezioni obbligatorie:
   - **Sezione 1 – Informazioni Generali**: tabella dei componenti del gruppo; dettagli riunione (data, ora, luogo, partecipanti, assenti, redattore, verificatore, versione).
   - **Sezione 2 – Ordine del Giorno**: elenco puntato degli argomenti pianificati.
   - **Sezione 3 – Diario della Riunione**: resoconto dettagliato articolato in sottosezioni che rispecchiano i punti dell'ordine del giorno.
   - **Sezione 4 – Decisioni Prese**: elenco numerato delle decisioni ufficiali deliberate.
   - **Sezione 5 – Attività da Svolgere (To-Do)**: tabella dei task con ruolo di riferimento.
   - **Sezione 6 – Definizione dei Ruoli (Preventivo)**: tabella con assegnazione dei ruoli e ore previste per lo sprint successivo.
2. Il Responsabile redige il verbale della riunione in cui **assume** il mandato (non di quella successiva che gestirà).
3. Il verbale viene verificato da un secondo membro del gruppo.
4. Per i verbali esterni, se richiesto dal proponente, il verbale viene sottoposto ad approvazione prima della pubblicazione.
5. Il verbale verificato (o approvato) viene caricato su GitHub e pubblicato tramite pipeline CI.

Strumenti

- **LaTeX / Overleaf**: redazione del verbale.
- **GitHub**: archiviazione e pubblicazione.

5 Processi Organizzativi

5.1 Gestione dei Ruoli

5.1.1 Attività 1 – Assegnazione e rotazione dei ruoli

Procedure

1. Il Responsabile di Progetto assegna i ruoli per ogni periodo di sprint, garantendo che ogni membro ricopra almeno una volta tutti i ruoli previsti nel corso del progetto.
2. **Fase Iniziale** (prime 6 settimane): rotazione settimanale per completare rapidamente il ciclo completo dei ruoli.
3. **Fase a Regime** (dalla settimana 7 in poi): rotazione ogni 2 settimane.
4. Lo switch formale dei ruoli avviene il **sabato alla mezzanotte** (tra venerdì e sabato).
5. Il Responsabile uscente documenta lo stato delle attività in corso per garantire continuità al successore.

Ruoli previsti e relative responsabilità

Responsabile di Progetto

- Redige i verbali delle riunioni, compresa la riunione interna di inizio mandato.
- Aggiorna GitHub Projects: inserisce e assegna i task emersi dalla riunione.
- Mantiene i contatti con proponente e committente.
- Redige ed espone il Diario di Bordo settimanale.
- Gestisce e coordina la riunione interna di sprint del venerdì.
- Aggiorna la sezione corrente del Piano di Progetto (attività svolte, risorse impiegate).
- Approva la documentazione ufficiale prodotta dal team.
- Contribuisce alla definizione degli obiettivi, delle scadenze e dell'allocazione delle risorse.

Amministratore

- Gestisce l'archiviazione e il versionamento di documentazione e codice sorgente.
- Amministra l'infrastruttura tecnologica e gli strumenti del team.
- Fornisce supporto tecnico e risolve malfunzionamenti segnalati.
- Automatizza i processi ricorrenti e ottimizza l'ambiente di lavoro.
- Controlla le versioni e le configurazioni del prodotto software.
- Redige e attua piani e procedure per la gestione della qualità.

Analista

- Raccoglie e analizza i requisiti del proponente, trasformandoli in specifiche formali.
- Definisce le specifiche funzionali$_G$ e tecniche$_G$ del prodotto.
- Modella concettualmente il sistema e classifica i requisiti per categorie logiche.
- Redige il documento *Analisi dei Requisiti*$_G$.
- Assicura che i requisiti siano espressi in modo chiaro, completo e non ambiguo.
- Collabora col Progettista per verificare la fattibilità dei requisiti identificati.

Progettista

- Progetta l'architettura software, privilegiando bassa dipendenza tra componenti e alta manutenibilità.
- Seleziona le tecnologie e i pattern architetturali$_G$ più appropriati.
- Produce i diagrammi UML (classi, sequenza, componenti).
- Verifica la sostenibilità economica e la manutenibilità della soluzione.
- Guida l'implementazione collaborando con Analisti e Programmatori.

Programmatore

- Implementa il codice rispettando le specifiche di progettazione e le norme di codifica definite nella sezione 5.
- Applica le best practices$_G$ di settore per garantire qualità e leggibilità.
- Risolve i problemi tecnici compatibili con l'architettura definita.
- Scrive test unitari per il proprio codice.
- Mantiene il codice ben documentato e correttamente versionato.

Verificatore

- Esamina i documenti su Overleaf prima del caricamento su GitHub.
- Esegue la code review delle pull request: controlla errori ortografici, sintattici, logici e problemi di build.
- Segnala non conformità con feedback dettagliati al produttore del lavoro.
- Redige la parte retrospettiva del Piano di Qualifica$_G$.
- Mantiene sorveglianza continua sulla qualità durante l'intero ciclo di vita del progetto.

Strumenti

- **GitHub Projects**: assegnazione e tracciamento dei ruoli per sprint.
- **Telegram**: comunicazione del cambio ruolo e passaggio di consegne.

5.2 Gestione delle Comunicazioni e degli Incontri

5.2.1 Attività 1 – Comunicazione interna

Procedure

1. Utilizzare **Telegram** per tutte le comunicazioni asincrone interne, organizzando i messaggi per topic (documentazione, riunioni, diario di bordo, ecc.).
2. Comunicare tempestivamente su Telegram eventuali ritardi, assenze o impedimenti.
3. Per questioni che richiedono confronto diretto, richiedere al Responsabile la convocazione di una riunione su Google Meet.

Strumenti

- **Telegram**: comunicazioni asincrone con funzionalità topic.
- **Google Meet**: riunioni sincrone interne.

5.2.2 Attività 2 – Comunicazione esterna

Procedure

1. Il Responsabile di Progetto gestisce tutte le comunicazioni ufficiali con proponente e committenti.
2. Utilizzare **email** (`swe.nightpro@gmail.com`) per comunicazioni formali e invio di documenti ufficiali.
3. Utilizzare **Telegram** per comunicazioni operative rapide che non richiedono formalità.
4. Organizzare le riunioni esterne tramite **Google Meet**; concordare data e ora con il proponente con anticipo sufficiente.
5. Redigere il verbale esterno al termine di ogni riunione con il proponente (vedi sezione 4.2.4).

Strumenti

- **Email** (`swe.nightpro@gmail.com`): comunicazioni formali esterne.
- **Telegram**: aggiornamenti operativi rapidi col proponente.
- **Google Meet**: riunioni esterne in modalità virtuale.

5.2.3 Attività 3 – Organizzazione delle riunioni interne

Procedure

1. La riunione principale si tiene ogni **venerdì mattina o pomeriggio**, in base alle disponibilità del guppo, ed è gestita dal Responsabile di Progetto in carica.
2. **Prima della riunione**:
   a. Il Responsabile prepara l'ordine del giorno con i punti da discutere.
   b. L'ordine del giorno viene comunicato ai membri su Telegram.
3. **Durante la riunione**:

   a. Il Responsabile guida la discussione, assicurando la trattazione di tutti i punti.
   b. Ogni membro riporta le attività svolte dall'ultimo incontro.
   c. Si discutono dubbi, problematiche e si prendono le decisioni necessarie.
   d. Si pianificano i task per il periodo successivo.

4. **Dopo la riunione**:
   a. Il Responsabile redige il verbale entro 48 ore.
   b. I nuovi task vengono inseriti su GitHub Projects con assegnatario e priorità.
   c. Il Diario di Bordo viene aggiornato con lo stato di avanzamento.
5. Qualsiasi membro può richiedere una riunione straordinaria al Responsabile; il Responsabile la organizza in base alla disponibilità del team.
6. I partecipanti si impegnano a partecipare puntualmente; le assenze devono essere comunicate su Telegram prima dell'inizio.

Strumenti

- **Google Meet**: piattaforma per le riunioni in modalità virtuale.
- **Telegram**: comunicazione dell'ordine del giorno e delle variazioni.
- **GitHub Projects**: inserimento dei task post-riunione.

5.2.4 Attività 4 – Organizzazione delle riunioni esterne

Procedure

1. Il Responsabile concorda con il proponente frequenza e modalità degli incontri.
2. Prima della riunione, il Responsabile prepara l'agenda e la condivide con il gruppo.
3. Durante la riunione, un membro prende nota degli argomenti trattati e delle decisioni prese.
4. Al termine, il Responsabile redige il verbale esterno secondo la struttura definita nella sezione 4.2.4.
5. Se richiesto dal proponente, il verbale viene sottoposto ad approvazione prima della pubblicazione.

Strumenti

- **Google Meet**: riunioni virtuali con proponente e committenti.
- **LaTeX / Overleaf**: redazione del verbale esterno.

5.3 Pianificazione e Tracciamento delle Attività

5.3.1 Attività 1 – Gestione del ciclo di vita dei task

Procedure

1. **Creazione**: il Responsabile crea il task su GitHub Projects compilando: titolo, descrizione, priorità, assegnatari, stima temporale e milestone$_G$ di riferimento.

2. **Assegnazione**: i task vengono distribuiti in base alle competenze e al carico di lavoro corrente; i task non assegnati possono essere presi in carico autonomamente.
3. **Esecuzione**:
   a. L'assegnatario aggiorna lo stato del task da "To Do" a "In Progress".
   b. Per task di documentazione: lavoro collaborativo su Overleaf.
   c. Per task di sviluppo: lavoro su branch dedicato (vedi sezione 3.2.3).
4. **Revisione**:
   a. Il Verificatore esamina il lavoro prodotto.
   b. Per documentazione: verifica su Overleaf prima del push su GitHub.
   c. Per codice: review della pull request con commenti su GitHub.
   d. Il Verificatore valida il task se conforme agli standard; altrimenti segnala le modifiche necessarie.
5. **Accettazione**:
   a. Il Responsabile approva il task completato.
   b. Per documentazione verificata: push su `main`.
   c. Per codice: merge del branch su `main`.
   d. Lo stato del task viene aggiornato a "Completato".

Strumenti

- **GitHub Projects**: creazione, assegnazione e tracciamento dei task.
- **GitHub**: gestione branch e pull request per i task di sviluppo.
- **Overleaf**: collaborazione sui task di documentazione.

5.4 Ottimizzazione dei Processi

5.4.1 Attività 1 – Applicazione del Ciclo di Deming (PDCA$_G$)

Procedure Il gruppo applica il **Ciclo di Deming**$_G$ (Plan-Do-Check-Act) a cadenza di sprint:

1. **Plan (Pianificare)**:
   a. Definire gli obiettivi dello sprint e i processi da eseguire.
   b. Identificare le metriche di riferimento e le soglie di accettazione.
   c. Pianificare le risorse e assegnare i task su GitHub Projects.
2. **Do (Fare)**:
   a. Eseguire le attività pianificate.
   b. Raccogliere dati sulle metriche di processo e di prodotto durante lo sprint.
3. **Check (Verificare)**:
   a. Al termine dello sprint, raccogliere i valori delle metriche.
   b. Confrontare i risultati con le soglie di accettazione definite nel Piano di Qualifica$_G$.
   c. Identificare aree di successo e opportunità di miglioramento.

   d. Documentare scostamenti e relative cause.

4. **Act (Agire)**:
   a. Formulare azioni correttive concrete per gli scostamenti rilevati.
   b. Tracciare le azioni correttive come task su GitHub Projects.
   c. Documentare le azioni nel verbale della riunione di sprint.
   d. Consolidare le modifiche validate come nuova baseline per il ciclo successivo.

Strumenti

- **GitHub Projects**: tracciamento delle azioni correttive.
- **LaTeX / Overleaf**: documentazione delle metriche nel Piano di Qualifica.
- **Verbali interni**: registrazione delle decisioni di miglioramento.

5.4.2 Attività 2 – Raccolta e analisi delle metriche

Procedure

1. Raccogliere al termine di ogni sprint i valori delle seguenti metriche, definite nel Piano di Qualifica$_G$:
   - **Metriche di processo** (metodologia EVM$_G$): scostamenti budget, varianze orarie, indici di performance temporale ed economica, proiezioni di completamento.
   - **Metriche di prodotto – documentazione**: indice Gulpease$_G$, correttezza ortografica, conformità redazionale.
   - **Metriche di prodotto – codice**: copertura dei test, complessità ciclomatica, densità di commenti, conformità alle convenzioni di stile.
2. Confrontare ogni valore con le soglie definite nel Piano di Qualifica:
   - **Valore preferibile**: obiettivo ottimale da perseguire.
   - **Valore accettabile**: soglia minima; al di sotto è richiesto un intervento correttivo.
3. Discutere i risultati nella riunione di sprint e avviare il passo **Act** del ciclo PDCA.

Strumenti

- **Piano di Qualifica**: definizione delle metriche e delle soglie di accettazione.
- **GitHub Projects**: tracciamento degli interventi correttivi.

5.5 Gestione dei Rischi

5.5.1 Attività 1 – Identificazione e analisi dei rischi

Procedure

1. Classificare ogni rischio con il codice `R[Tipo][Indice]`, dove:
   - **RT**: rischi tecnologici (strumenti, tecnologie, infrastrutture).
   - **RO**: rischi organizzativi (gestione, comunicazione, coordinamento).
   - **RP**: rischi personali (disponibilità, competenze, impegni individuali).

2. Per ogni rischio identificato, valutare:
   a. **Probabilità di occorrenza**: bassa / media / alta.
   b. **Grado di pericolosità**: basso / medio / alto.
3. Determinare la priorità del rischio in base alla combinazione di probabilità e pericolosità.
4. Definire per ogni rischio una strategia di mitigazione preventiva e un piano di contingenza.
5. Documentare l'elenco dei rischi nel *Piano di Progetto*$_{\mathrm{G}}$.

Strumenti

- **Piano di Progetto**: registro ufficiale dei rischi e delle strategie di mitigazione.
- **Telegram / riunioni interne**: segnalazione di nuovi rischi emergenti.

5.5.2 Attività 2 – Monitoraggio dei rischi

Procedure

1. Ad ogni riunione di sprint, il Responsabile verifica lo stato dei rischi già identificati.
2. Se un rischio si concretizza, si attua il piano di contingenza definito e si documenta l'evento nel verbale.
3. Identificare nuovi rischi emersi durante lo sprint e aggiungerli al Piano di Progetto con classificazione e strategia.
4. Valutare l'efficacia delle misure di mitigazione adottate nei cicli precedenti.

Strumenti

- **Piano di Progetto**: aggiornamento del registro dei rischi.
- **GitHub Projects**: tracciamento delle azioni di mitigazione come task.

5.6 Diario di Bordo

5.6.1 Attività 1 – Preparazione e svolgimento

Procedure

1. **Preparazione** (entro il giorno precedente l'esposizione):
   a. Il Responsabile in carica prepara 2–3 diapositive di contenuto.
   b. Le diapositive devono includere: sintesi delle attività svolte nel periodo; difficoltà tecniche o organizzative incontrate; dubbi aperti e richieste di chiarimento; stato di avanzamento rispetto alla pianificazione.
   c. La durata massima dell'esposizione è **4 minuti**.
2. **Svolgimento**:
   a. Il Responsabile espone condividendo lo schermo nella stanza Zoom designata.
   b. Al termine delle esposizioni di tutti i gruppi si partecipa alla discussione collettiva.

Strumenti

- **Strumento di presentazione concordato dal gruppo**: creazione delle diapositive.
- **Zoom**: piattaforma di esposizione del Diario di Bordo.

5.7 Progettazione dell'Interfaccia Utente

5.7.1 Attività 1 – Prototipazione e validazione UI

Procedure

1. **Creazione dei mockup**:
   a. Definire su Penpot$_G$ le schermate principali del prodotto e i flussi di navigazione.
   b. Coprire tutti i requisiti funzionali obbligatori che impattano sull'interfaccia.
2. **Revisione interna**:
   a. Presentare i mockup durante la riunione di sprint.
   b. Raccogliere feedback da tutti i membri del gruppo.
   c. Applicare le modifiche concordate prima della validazione esterna.
3. **Validazione con il proponente**:
   a. Presentare i prototipi al proponente in una riunione esterna.
   b. Documentare nel verbale esterno le conferme o le richieste di modifica.
   c. Iterare il processo (passi 1–3) fino all'approvazione del proponente.
4. **Passaggio all'implementazione**:
   a. I mockup approvati costituiscono il riferimento vincolante per lo sviluppo del frontend.
   b. Qualsiasi scostamento dai mockup approvati deve essere motivato e concordato col Responsabile.

Strumenti

- **Penpot**: piattaforma open source per la creazione di mockup e prototipi interattivi.
- **Google Meet**: presentazione dei prototipi al proponente.

5.8 Sviluppo delle Competenze

5.8.1 Attività 1 – Formazione individuale e condivisione delle conoscenze

Procedure

1. Ogni membro del gruppo è responsabile dell'apprendimento autonomo delle tecnologie e degli strumenti necessari al proprio ruolo corrente.
2. Le aree di formazione prioritarie sono:
   - Linguaggio LaTeX per la produzione documentale.
   - Linguaggi di programmazione, librerie e framework richiesti dal capitolato C8.
   - Strumenti e metodologie dell'ambiente di lavoro del proponente.

3. Il membro che acquisisce una competenza rilevante per il gruppo la condivide durante la riunione di sprint o tramite Telegram.
4. La rotazione periodica dei ruoli (sezione 5.1) garantisce l'acquisizione di esperienza pratica in tutti i ruoli previsti.

Strumenti

- **Risorse autonome**: documentazione ufficiale, tutorial, corsi online scelti dal singolo membro.
- **Telegram**: condivisione rapida di risorse e conoscenze tra i membri.
- **Google Meet**: sessioni di knowledge sharing durante le riunioni di sprint.